

DEVICE DRIVER ROADMAP — Tailored for Your Profile

✓ You already know:

Embedded C
RTOS & Bare-metal
Linux BSP customization
UART/SPI/I2C
Debuggers: JTAG, oscilloscopes, logic analyzer
Working with SBCs, GPUs, PSUs
Linux-based systems and Yocto

So you can skip beginner embedded topics.
Start directly from Linux Kernel + Driver Basics.

■ STAGE 1 — Strong Kernel Fundamentals (2–3 weeks)

Kernel architecture (user vs kernel space)
System calls, VFS, kernel modules
printk, dmesg, module parameters
Building/compiling kernel
Kernel configuration & Kconfig basics

Why this is easy for you:

Because you already worked on BSP and Yocto — so this step will be fast.

■ STAGE 2 — Character Device Drivers (MOST IMPORTANT) (1.5–2 months)

register_chrdev_region
cdev_init, cdev_add
/dev entries

File operations:

- open
- read
- write
- release
- ioctl

copy_to_user / copy_from_user

mutex, semaphore, spinlock, completion, wait queues

Mini Projects:

- ✓ Virtual memory-based driver
- ✓ IOCTL-based LED control driver
- ✓ Circular buffer driver
- ✓ Blocking/non-blocking read/write

■ STAGE 3 — Linux Kernel Memory Management (2–3 weeks)

kmalloc vs vmalloc
slab allocator
Linux memory mapping
mmap for drivers
Accessing MMIO registers (ioremap)
Understanding device memory vs normal memory

Project:

- ✓ mmap()-based driver

■ STAGE 4 — Hardware-Specific Drivers (2–3 months)

1. GPIO Driver

gpio APIs, interrupt handling, debouncing

2. I2C Driver

adapter/client model, probe/remove, regmap

3. SPI Driver

spidev, sync/async transfers

4. Interrupt Handlers

request_irq, ISR, tasklets, workqueues

Projects:

- ✓ I2C temperature sensor driver (LM75)
- ✓ SPI OLED driver (SSD1306)
- ✓ GPIO interrupt driver

■ STAGE 5 — Platform Drivers + Device Tree (2 months)

device tree structure
compatible strings
device tree binding documentation
platform_device vs platform_driver
device_initcall

Projects:

- ✓ DT entry + platform driver for LED/GPIO
- ✓ Platform driver for ADC/sensor

■ STAGE 6 — Advanced Driver Domains (Optional)

Automotive/Defence:

CAN driver, MIL-STD-1553 stack

Kernel Engineering:

Power management, regulator subsystem, sysfs/procfs, kernel timers

High-performance systems:

DMA API, PCIe, MSI-X interrupts

■ STAGE 7 — Portfolio Projects (VERY IMPORTANT)

Build at least 4 drivers:

- ✓ Character driver with IOCTL
- ✓ GPIO interrupt driver
- ✓ I2C sensor driver (DT-based)
- ✓ SPI display driver (SSD1306/ST7735)

Optional:

CAN driver, DMA demo, regulator/clock framework driver

Upload to GitHub for resume impact.